Setting Up SkyNYC

From a clean machine to a running ingestion pipeline, in about forty-five minutes.

Covers: prerequisites · API access · configuration · first start · verification | August 2026

---

1. What you are about to stand up

SkyNYC watches the New York terminal area — JFK, LaGuardia, and Newark — and measures, in real time, how weather degrades airport operations. It pulls live aircraft positions from the OpenSky Network and official observations and alerts from the National Weather Service, streams them through Kafka, and lands them in Postgres for analysis and a live dashboard.

Everything runs in Docker — on your machine while you develop, and on a small rented server for continuous collection. Two cloud accounts exist in the whole project: one for that server and one for the object storage that holds the raw archive; both are the subject of guide 3. This guide sets up the local development environment. By the end you will have:

- Kafka, Postgres, and Grafana running as containers, with the database schema applied.
- Three Kafka topics carrying live aircraft states, weather observations, and weather alerts.
- Two producers polling the live APIs, and a consumer writing weather into Postgres.
- A smoke test proving all of it against the real services — not mocks.

---

2. Before you start

Work through this checklist first. Every support question I have ever fielded on this stack traces back to one of these three items.

ITEM	WHAT TO DO	WHY IT MATTERS
Docker Desktop	Install it, then open *Settings → Resources* and allocate at least **8 GB of RAM** and 4 CPUs.	The full stack idles around 6 GB. Under-allocated Docker fails in confusing ways — containers restart in loops with no obvious error.
git	Any recent version. Confirm with `git --version`.	You will clone the repository and, later, commit dashboard changes.
An email address	Pick one you are comfortable sending in an HTTP header.	The National Weather Service requires a contact address in every request. It is their only condition for free access — respect it.

Confirm the Docker Compose plugin is the v2 syntax: `docker compose version` (a space, not a hyphen).

---

3. Create your OpenSky API client

OpenSky is a research network of volunteer-run ADS-B receivers. A free account gives you 4,000 request credits per day — far more than this pipeline uses. The setup is ten minutes, and you only ever do it once.

1. Go to `opensky-network.org` and sign up. No card, no payment. Verify your email.
2. Log in and open *My OpenSky → Account*.
3. In the API client section, click **Create new API client**.
4. Copy the `client_id` and `client_secret` somewhere safe *immediately*.

> **TREAT THE SECRET LIKE A PASSWORD**
>
> It may only be shown once. If you lose it — or ever paste it into a screenshot, a chat, or a commit — do not try to un-lose it. Go back to the Account page, delete the client, create a new one, and update your configuration. Rotation is a two-minute fix; a leaked credential is not.

---

4. Configure the environment

All configuration lives in a single `.env` file at the repository root. The repository ships a template with every key name and no values:

```
cd skynyc
cp .env.example .env
```

Open `.env` and fill in three values. Leave the rest at their defaults.

KEY	VALUE
`OPENSKY_CLIENT_ID`	From step 3.
`OPENSKY_CLIENT_SECRET`	From step 3.
`CONTACT_EMAIL`	The address you chose in step 2. It is sent as `User-Agent: skynyc (you@example.com)` on every weather request.

> **THE RULE ABOUT .ENV**
>
> `.env` is listed in `.gitignore` and must stay there. Credentials never go into version control, into code, or into logs. If you ever see a secret printed in a terminal, treat it as leaked and rotate it.

---

5. Start the infrastructure

```
make up
```

This starts Kafka, Postgres, and Grafana in the background. Three things happen on a first boot that are worth understanding:

- **Postgres applies the schema itself.** Scripts in the image's init directory run automatically — but *only when the data volume is empty*, which is to say only on the very first boot. Remember this: editing that schema file later and restarting does nothing. Schema changes after today are applied as separate, numbered migration files.
- **Kafka forms a single-node cluster.** Give it up to thirty seconds to report healthy.
- **Grafana comes up on** `localhost:3000` (sign in with `admin` / `admin` and change the password when prompted). The dashboard itself arrives in a later milestone; for now an empty Grafana is correct.

Check state before moving on — this is a habit, not a suggestion:

```
docker compose ps
```

Kafka should show *(healthy)*, Postgres *(healthy)*, Grafana *Up*. A container stuck restarting almost always means a missing `.env` value — check its logs with `docker compose logs <service>`.

6. Create the topics

```
make topics
```

This creates three topics and is safe to run repeatedly:

TOPIC	CARRIES	RETENTION
`adsb.states.raw`	One message per aircraft per poll, keyed by the aircraft's transponder address	7 days
`wx.observations`	Parsed station observations, keyed by station	30 days
`wx.alerts`	Active weather alerts, keyed by airport	30 days

Retention here is not housekeeping trivia — the states topic *is* the system's replay history. The upstream API serves at most one hour of the past, so those seven days of retained messages are the only archive that exists. Nobody deletes Kafka volumes casually on this project.

7. Run the smoke test

```
make smoke
```

The smoke test proves the three things everything else depends on, against the live services:

1. **Authentication works** — it exchanges your client credentials for a bearer token and shows its 30-minute expiry.
2. **Aircraft data flows** — one authenticated request over the New York bounding box, reporting how many aircraft are up right now. Expect anywhere from ~40 on a quiet night to ~250 in an evening push.
3. **Weather data flows** — the latest JFK observation, which must be under two hours old, with every field's unit code printed.

All three checks must print a checkmark. This costs exactly one API credit of your 4,000 — the budget is examined in the operations guide, but know from day one that credits are a shared, finite resource.

8. Start ingestion

```
make ingest
```

This builds one small Python image and starts the three long-running services:

SERVICE	WHAT IT DOES	CADENCE
`producer-opensky`	Polls aircraft states over the NYC box, publishes one message per aircraft	Every 30 s
`producer-weather`	Polls observations and active alerts for JFK, LGA, EWR	Obs 5 min · alerts 2 min
`consumer-weather`	Writes weather messages into Postgres	Continuous

Give it a minute, then confirm all three are doing real work:

```
docker compose logs --tail 5 producer-opensky   # expect: poll ok n_states=… credits_remaining=…
docker compose logs --tail 5 producer-weather   # expect: obs published / alerts active=…
docker compose logs --tail 5 consumer-weather   # expect: upserted wx.observations key=KJFK …
```

9. Verify your work

Run each check; every row must pass before you call the setup finished. Do not skip this because the logs "looked fine" — the checks below are the definition of fine.

CHECK	COMMAND	PASS CONDITION
All services up	`docker compose ps`	Six containers, none restarting
States flowing	`make lag`	Weather consumer listed with lag 0 or single digits
Credit budget	`make credits`	Remaining count printed; projection under budget
Weather landing	`make psql` then `SELECT station, obs_ts, flight_category FROM weather_obs;`	One or more rows per station, timestamps recent

10. When something goes wrong

SYMPTOM	CAUSE	FIX
Smoke check 1 fails with HTTP 401	Wrong or mistyped client credentials	Re-check both values in `.env` against the OpenSky Account page; recreate the client if the secret is lost
Smoke check 3 fails with HTTP 403	Missing or blank contact email	Set `CONTACT_EMAIL` in `.env` — the weather service rejects requests without an identifying User-Agent
Zero aircraft between about 03:00 and 05:00 local	Real overnight traffic lull	Nothing — this is genuine data, not an outage. Re-run in daytime if it worries you
A container restarts in a loop	Almost always a missing `.env` value	`docker compose logs <service>` and read the first error, not the last
Schema missing in Postgres	The database volume existed before this checkout	Ask before destroying anything — but on a genuinely fresh setup with no data worth keeping, `docker compose down -v` then `make up` re-runs init

11. A word on being a good API citizen

Both data sources are free, and both stay pleasant to use only if we are polite. The pipeline budgets about 2,880 of your 4,000 daily OpenSky credits — your own experiments come out of the same bucket, so never loop a states request by hand in a terminal. The weather service asks only for the contact header and restraint; roughly two requests a minute across all stations is well within courtesy. When you publish anything built on this data, cite the OpenSky Network — it is the stated condition of their free research access.